Poora flow — dry run

```
main() mein:
↓
NotificationEngine engine banaya  ← ye Subject hai

EmailChannel email banaya         ← Observer
SMSChannel sms banaya             ← Observer
PushChannel push banaya           ← Observer

engine.attach(&email)
engine.attach(&sms)
engine.attach(&push)

observers list:
[&email, &sms, &push]
```

Ab order status badla:

```
engine.notify("Order Placed")
↓
notifyAll() chala
↓
loop:
  email.update("Order Placed")
  sms.update("Order Placed")
  push.update("Order Placed")
```

Ab Decorator wala part:

```
"Order Placed"
↓
PlainMessage("Order Placed")         → "Order Placed"
↓
TimestampDecorator(PlainMessage)     → "Order Placed -- 10:45 PM"
↓
UrgentDecorator(TimestampDecorator)  → "[URGENT] Order Placed -- 10:45 PM"
```

Ye wrapped message har channel ko jaayega.

Final output dikhega kuch aisa:

```
[EMAIL] : [URGENT] Order Placed -- 10:45 PM
[SMS]   : [URGENT] Order Placed -- 10:45 PM
[PUSH]  : [URGENT] Order Placed -- 10:45 PM

[EMAIL] : [URGENT] Out for Delivery -- 10:52 PM
[SMS]   : [URGENT] Out for Delivery -- 10:52 PM

[EMAIL] : [URGENT] Delivered -- 11:10 PM
```

SMS detach ho gaya beech mein — isliye sirf Email aaya last mein.

Classes ka structure — ek nazar mein

```
IMessage                    ← Decorator ka base
  PlainMessage              ← base message
  MessageDecorator          ← wrapper base
    UrgentDecorator         ← [URGENT] add karta hai
    TimestampDecorator      ← timestamp add karta hai

IObserver                   ← Observer ka base
  EmailChannel              ← email print karta hai
  SMSChannel                ← sms print karta hai
  PushChannel               ← push print karta hai

NotificationEngine          ← Subject
  attach()
  detach()
  notifyAll()
  notify(string event)      ← message decorate karke notifyAll() call karta hai
```

Kya bana rahe hain — Notification Engine

Real life scenario — **Zomato order place hua.**

Jab order ka status badlega — system automatically saare registered channels ko notify karega. Aur message bhejna se pehle usse decorate karega — timestamp lagayega, urgent tag lagayega.

Poora flow — dry run

main() mein:
↓
NotificationEngine engine banaya ← ye Subject hai

EmailChannel email banaya ← Observer
SMSChannel sms banaya ← Observer
PushChannel push banaya ← Observer

engine.attach(&email)
engine.attach(&sms)
engine.attach(&push)

observers list:
[&email, &sms, &push]

Ab order status badla:

engine.notify("Order Placed")
↓
notifyAll() chala
↓
loop:
  email.update("Order Placed")
  sms.update("Order Placed")
  push.update("Order Placed")

Ab Decorator wala part:

"Order Placed"
↓
PlainMessage("Order Placed") → "Order Placed"
↓
TimestampDecorator(PlainMessage) → "Order Placed -- 10:45 PM"
↓
UrgentDecorator(TimestampDecorator) → "[URGENT] Order Placed -- 10:45 PM"

Ye wrapped message har channel ko jaayega.

Final output dikhega kuch aisa:

[EMAIL] : [URGENT] Order Placed -- 10:45 PM
[SMS] : [URGENT] Order Placed -- 10:45 PM

[PUSH] : [URGENT] Order Placed -- 10:45 PM

[EMAIL] : [URGENT] Out for Delivery -- 10:52 PM
[SMS]   : [URGENT] Out for Delivery -- 10:52 PM

[EMAIL] : [URGENT] Delivered -- 11:10 PM

SMS detach ho gaya beech mein — isliye sirf Email aaya last mein.

Classes ka structure — ek nazar mein

```
IMessage                ← Decorator ka base
  PlainMessage          ← base message
  MessageDecorator      ← wrapper base
    UrgentDecorator     ← [URGENT] add karta hai
    TimestampDecorator  ← timestamp add karta hai

IObserver               ← Observer ka base
  EmailChannel          ← email print karta hai
  SMSChannel            ← sms print karta hai
  PushChannel           ← push print karta hai

NotificationEngine      ← Subject
  attach()
  detach()
  notifyAll()
  notify(string event)  ← message decorate karke notifyAll() call karta hai
```